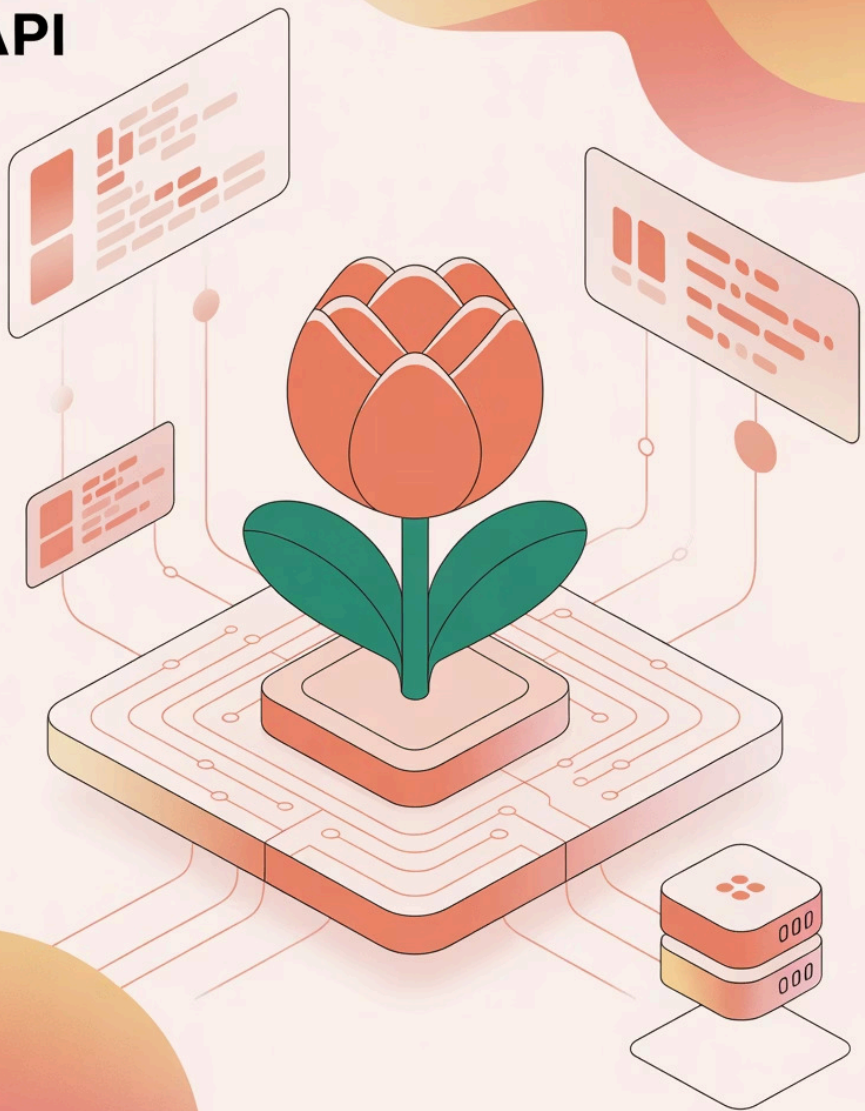


REST
API



Recibiendo datos en endpoints REST

Video #4 - Java con Spring Boot de 0 a Pro

APIs REST con Spring Boot

Java Spring Boot Tutorial

¿Qué aprenderás hoy?

01

Recibir datos del cliente

Comprenderás las diferentes formas de capturar información enviada por el cliente a tus endpoints REST

02

@PathVariable en acción

Aprenderás a extraer valores dinámicos directamente desde la URL de la petición

03

@RequestParam para queries

Dominarás el manejo de parámetros opcionales y filtros en tus endpoints

04

@RequestBody para JSON

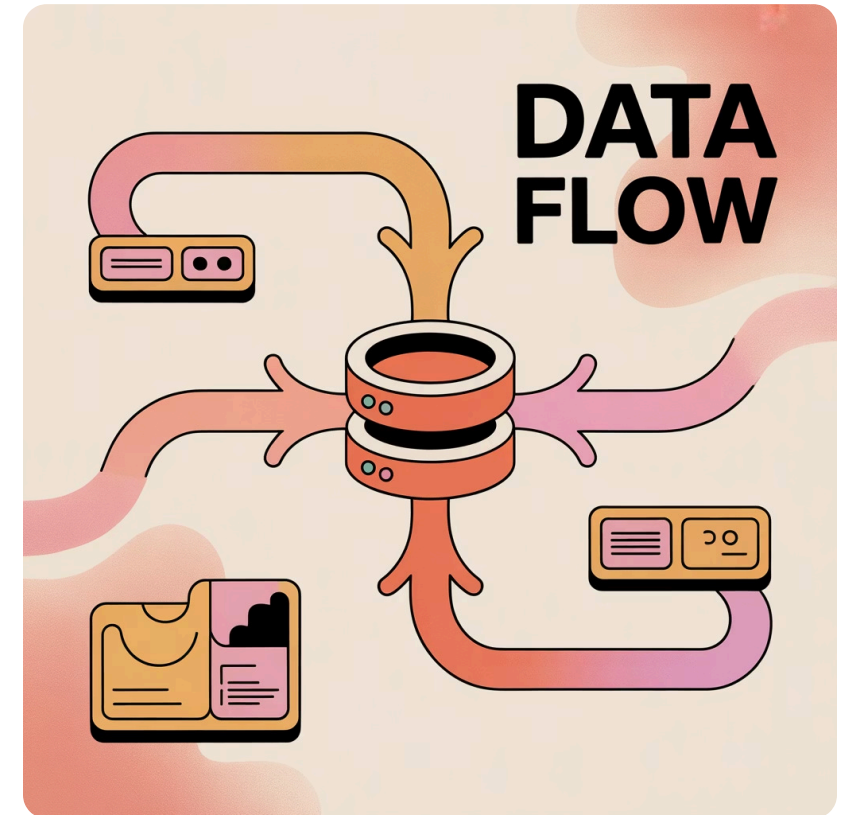
Entenderás cómo recibir y procesar objetos JSON completos enviados en el cuerpo de la petición

¿Por qué necesitamos recibir datos?

Los endpoints estáticos tienen utilidad limitada en aplicaciones reales. Para crear APIs verdaderamente funcionales, necesitamos que nuestros endpoints puedan:

- Procesar información específica del usuario
- Filtrar resultados según criterios dinámicos
- Crear, actualizar y eliminar recursos
- Personalizar respuestas basadas en parámetros

Sin esta capacidad, nuestras APIs serían completamente rígidas e inútiles para la mayoría de casos de uso empresariales.



@PathVariable: Datos en la URL

¿Qué es?

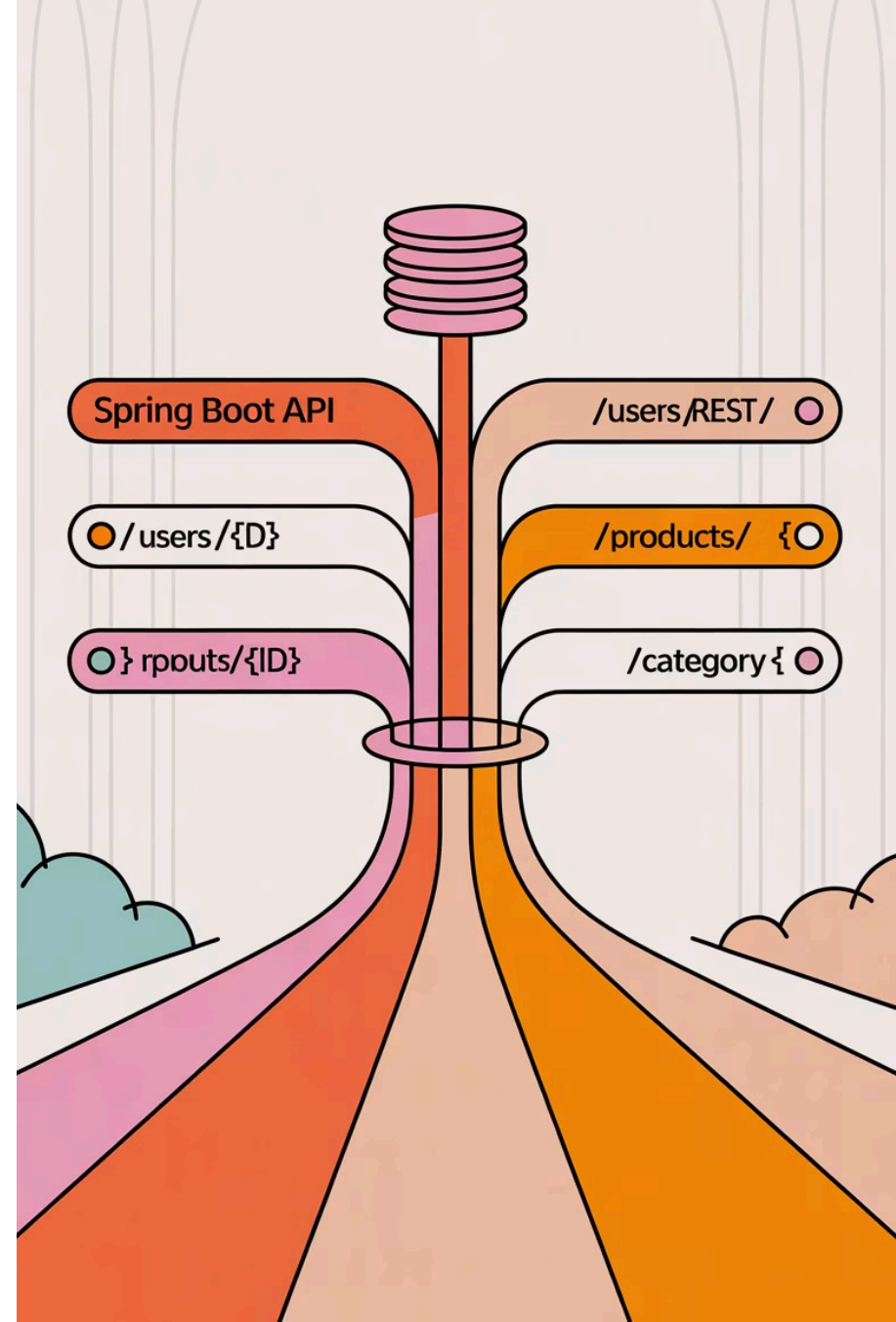
Extrae valores dinámicos directamente de la ruta de la URL. Ideal para identificadores únicos como IDs.

Sintaxis básica

```
@GetMapping("/usuarios/{id}")
public Usuario obtenerUsuario(
    @PathVariable Long id) {
    return
    usuarioService.buscar(id);
}
```

Ejemplo de uso

GET /usuarios/123 → el valor 123 se captura automáticamente en la variable 'id'



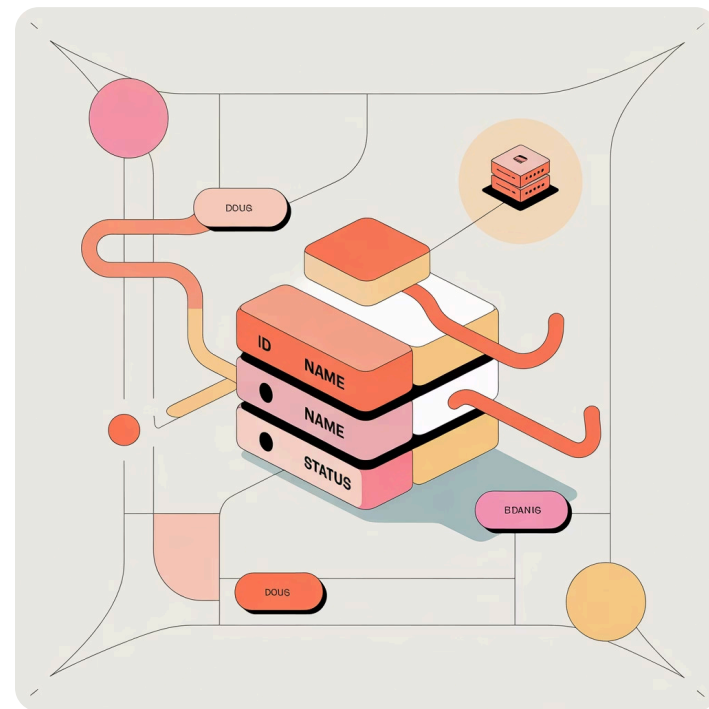
@RequestParam: Parámetros de consulta

Características principales

- Se envían después del símbolo ? en la URL
- Perfectos para filtros y búsquedas
- Pueden ser opcionales con valores por defecto
- Múltiples parámetros separados por &

Ejemplo práctico

```
@GetMapping("/productos")
public List<Producto> listar(
    @RequestParam(required = false)
    String categoria,
    @RequestParam(defaultValue = "0")
    int pagina) {
    return productoService
        .buscar(categoria, pagina);
}
```



URL de ejemplo:

GET /productos?categoria=electronica&pagina=2

@RequestBody: Recibiendo JSON

Propósito principal

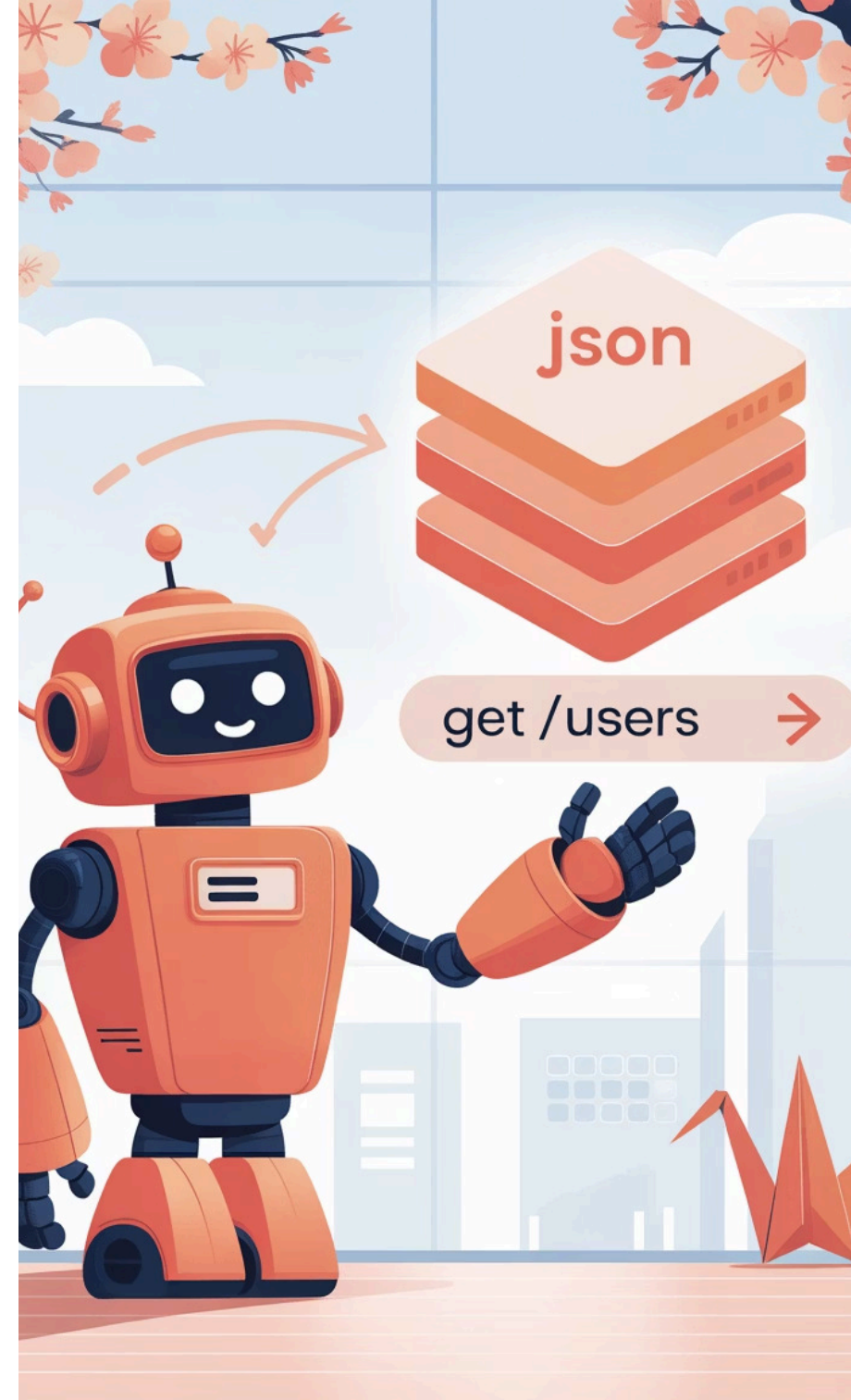
Recibe objetos JSON completos enviados en el cuerpo de peticiones POST, PUT o PATCH. Spring Boot convierte automáticamente el JSON a objetos Java.

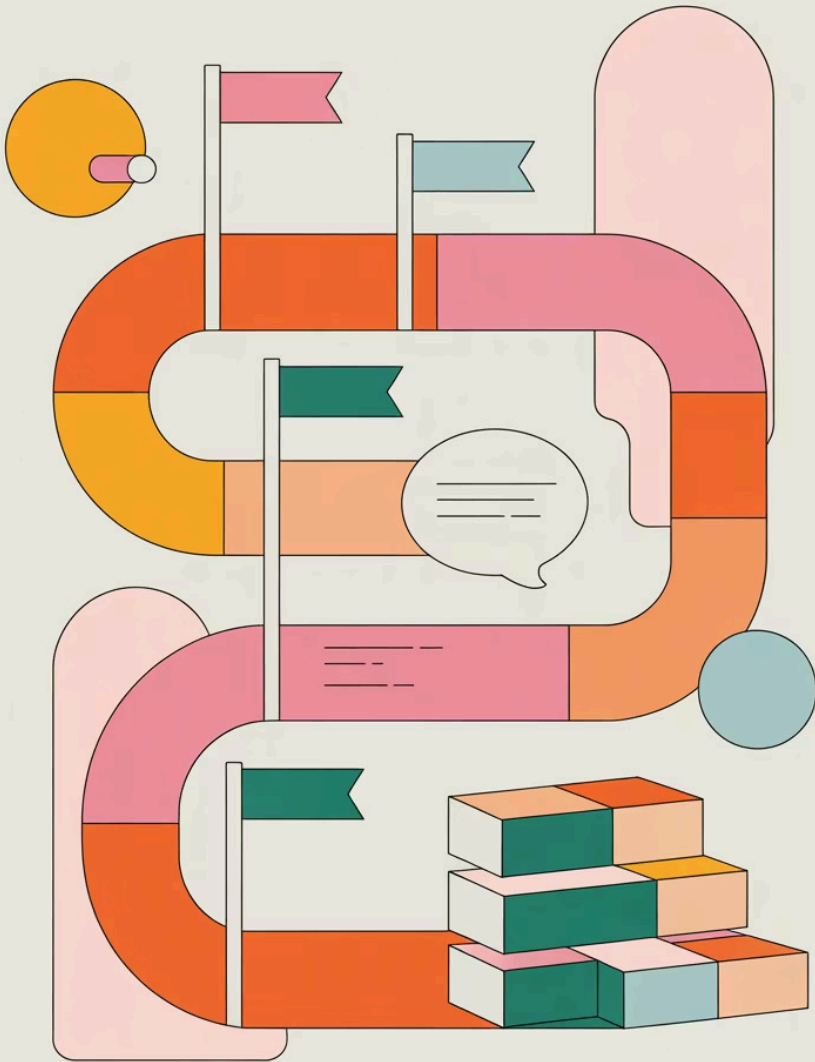
Ejemplo de implementación

```
@PostMapping("/usuarios")
public Usuario crear(
    @RequestBody Usuario
    nuevoUsuario) {
    return
    usuarioService.guardar(nuevoU
    suario);
}
```

JSON de entrada

```
{
  "nombre": "Juan Pérez",
  "email": "juan@email.com",
  "edad": 30
}
```





Comparación: Ruta vs Query vs Body

@PathVariable

Ubicación: En la ruta

Ejemplo: /usuarios/123

Uso típico: IDs, identificadores únicos

Requerido: Siempre obligatorio

@RequestParam

Ubicación: Query string

Ejemplo: /productos?categoria=tech

Uso típico: Filtros, paginación, búsquedas

Requerido: Puede ser opcional

@RequestBody

Ubicación: Cuerpo HTTP

Ejemplo: JSON completo

Uso típico: Crear/actualizar recursos

Requerido: Generalmente obligatorio

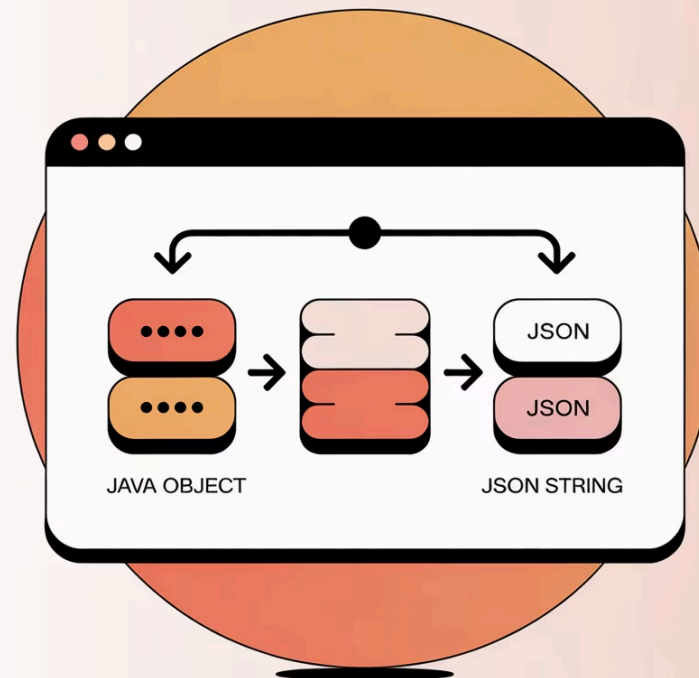
Serialización automática JSON ↔ Java

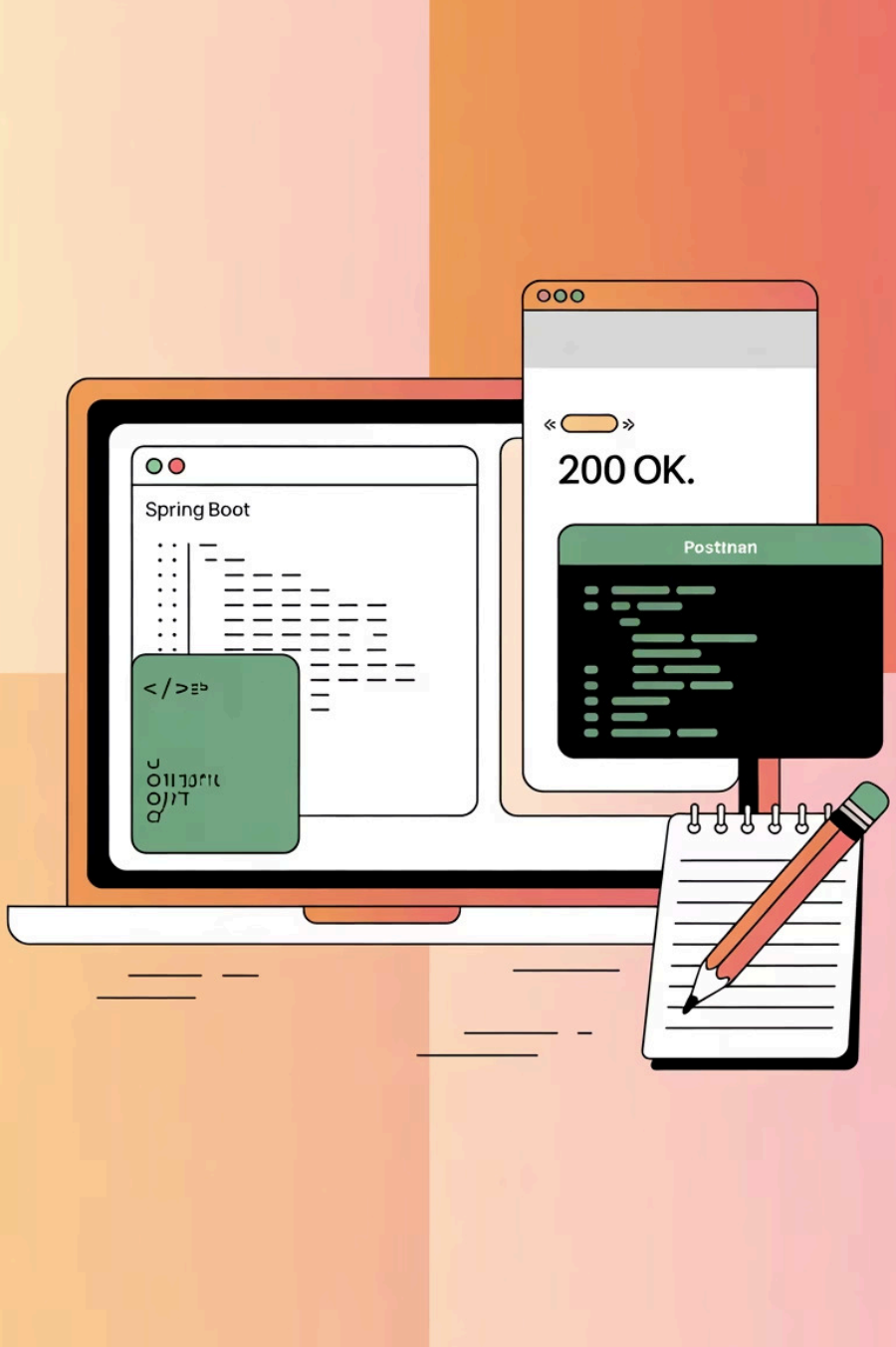
¿Cómo funciona la magia?

Spring Boot incluye Jackson por defecto, una librería que se encarga de:

- **Deserialización:** Convertir JSON entrante a objetos Java
- **Serialización:** Transformar objetos Java a JSON de respuesta
- **Mapeo automático:** Coincidencia por nombres de propiedades
- **Validación:** Verificar tipos de datos y estructura

✓ No necesitas escribir código adicional para estas conversiones.
¡Spring Boot lo hace automáticamente!





Checklist para la demostración

1 Preparar el entorno

Abrir IDE con proyecto Spring Boot creado en video anterior

Verificar que el servidor se ejecute correctamente en puerto 8080

2 Crear controlador completo

Implementar endpoints con `@PathVariable`, `@RequestParam` y `@RequestBody`

Incluir ejemplos realistas con clase `Usuario` o `Producto`

3 Configurar Postman

Crear colección de requests para probar cada tipo de endpoint

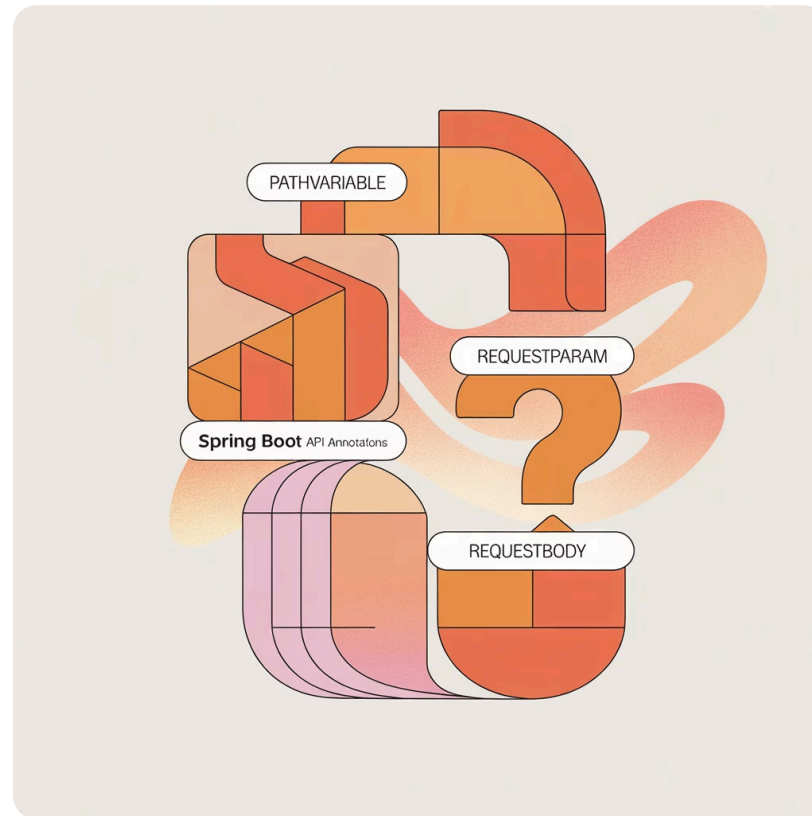
Preparar JSONs de ejemplo para peticiones POST

Resumen ejecutivo

Lo que dominaste hoy:

- **@PathVariable**: Extraer IDs y valores de la URL
- **@RequestParam**: Manejar filtros y parámetros opcionales
- **@RequestBody**: Recibir objetos JSON complejos
- **Serialización**: Conversión automática JSON ↔ Java

📌 Con estas tres anotaciones ya puedes crear APIs REST completamente funcionales que manejen cualquier tipo de entrada de datos.





Próximos pasos en tu journey

1

Video #5: Validación de datos

Aprenderás a validar automáticamente los datos entrantes usando Bean Validation (@Valid, @NotNull, etc.)

2

Video #6: Manejo de errores

Implementarás respuestas HTTP apropiadas y manejo elegante de excepciones con @ExceptionHandler

3

Video #7: Conectando con base de datos

Integrarás Spring Data JPA para persistir y consultar datos desde una base de datos real

¡Nos vemos en el próximo video! 🚀



¡Ahora es tu turno!

Pon en práctica lo aprendido y sigue construyendo APIs REST increíbles con Spring Boot.

¡Nos vemos en el siguiente módulo!